Monitoring Task

Task Description:

Install Prometheus and Grafana on a Linux EC2 machine, connect Prometheus to Grafana, and create a dashboard to view metrics.

File Architecture:

Creating the EC2 machine through Terraform and installing the Prometheus and Grafana using the UserData.

```
GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$ tree
.
├── main.tf
├── outputs.tf
├── terraform.tfstate
├── terraform.tfstate.backup
├── user_data.sh
└── variables.tf

1 directory, 6 files
GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$
```

Outputs.tf

```
output "public_ip" {
  value = aws_instance.monitoring_server.public_ip
}

output "grafana_url" {
  value = "http://${aws_instance.monitoring_server.public_ip}:3000"
}

output "prometheus_url" {
  value = "http://${aws_instance.monitoring_server.public_ip}:9090"
}
~
```

Variables.tf

```
variable "region" {
  default = "ap-south-1"
}
```

Main.tf

```
provider "aws" {
  region = var.region
}

# ------------------------------
# Get Latest Ubuntu 22.04 AMI
# ------------------------------
data "aws_ami" "ubuntu" {
  most_recent = true

  owners = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

resource "aws_key_pair" "generated_key" {
  key_name   = "terraform-key"
  public_key = file("~/.ssh/id_rsa.pub")
}


# ------------------------------
# Security Group
# ------------------------------
resource "aws_security_group" "monitoring_sg" {
  name        = "monitoring-sg"
  description = "Allow Prometheus, Grafana, Node Exporter"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 9090
    to_port     = 9090
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 3000
    to_port     = 3000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 9100
    to_port     = 9100
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# ------------------------------
# EC2 Instance
# ------------------------------
resource "aws_instance" "monitoring_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  key_name = aws_key_pair.generated_key.key_name

  security_groups = [aws_security_group.monitoring_sg.name]

  user_data = file("user_data.sh")
```

User_data.sh

```
#!/bin/bash
set -xe

exec > /var/log/user-data.log 2>&1

echo "STARTING SETUP"

# Update system
apt update -y
apt install -y wget curl tar software-properties-common

cd /opt

# ========================
# Install Node Exporter
# ========================

NODE_VERSION=$(curl -s https://api.github.com/repos/prometheus/node_exporter/releases/latest | grep tag_name | cut -d '"' -f 4 | sed 's/v//')

wget https://github.com/prometheus/node_exporter/releases/download/v${NODE_VERSION}/node_exporter-${NODE_VERSION}.linux-amd64.tar.gz

tar -xvf node_exporter-${NODE_VERSION}.linux-amd64.tar.gz

cp node_exporter-${NODE_VERSION}.linux-amd64/node_exporter /usr/local/bin/

# Create service
cat <<EOF > /etc/systemd/system/node_exporter.service
[Unit]
Description=Node Exporter
After=network.target

[Service]
ExecStart=/usr/local/bin/node_exporter
Restart=always

[Install]
WantedBy=multi-user.target
EOF

# ========================
# Install Prometheus
# ========================

PROM_VERSION=$(curl -s https://api.github.com/repos/prometheus/prometheus/releases/latest | grep tag_name | cut -d '"' -f 4 | sed 's/v//')

wget https://github.com/prometheus/prometheus/releases/download/v${PROM_VERSION}/prometheus-${PROM_VERSION}.linux-amd64.tar.gz

tar -xvf prometheus-${PROM_VERSION}.linux-amd64.tar.gz

cp prometheus-${PROM_VERSION}.linux-amd64/prometheus /usr/local/bin/
cp prometheus-${PROM_VERSION}.linux-amd64/promtool /usr/local/bin/
```

```
mkdir -p /etc/prometheus
mkdir -p /var/lib/prometheus

cat <<EOF > /etc/prometheus/prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets: ["localhost:9100"]
EOF

# Create service
cat <<EOF > /etc/systemd/system/prometheus.service
[Unit]
Description=Prometheus
After=network.target

[Service]
ExecStart=/usr/local/bin/prometheus \
--config.file=/etc/prometheus/prometheus.yml \
--storage.tsdb.path=/var/lib/prometheus
Restart=always

[Install]
WantedBy=multi-user.target
EOF

# ========================
# Install Grafana
# ========================

wget -q -O - https://packages.grafana.com/gpg.key | apt-key add -
echo "deb https://packages.grafana.com/oss/deb stable main" > /etc/apt/sources.list.d/grafana.list

apt update -y
apt install -y grafana

# ========================
# Start Services
# ========================

systemctl daemon-reload

systemctl enable node_exporter
systemctl start node_exporter

systemctl enable prometheus
systemctl start prometheus

systemctl enable grafana-server
```

Building an EC2 Instance and deploying Prometheus and Grafana using User Data script:

```
GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$ ls
main.tf  outputs.tf  terraform.tfstate  terraform.tfstate.backup  user_data.sh  variables.tf
GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.39.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$ terraform validate
Success! The configuration is valid.

GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$ terraform plan
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 1s [id=ami-095cc90aa5ddff518]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.monitoring_server will be created
  + resource "aws_instance" "monitoring_server" {
      + ami                                  = "ami-095cc90aa5ddff518"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "terraform-key"
      + monitoring                           = (known after apply)
```

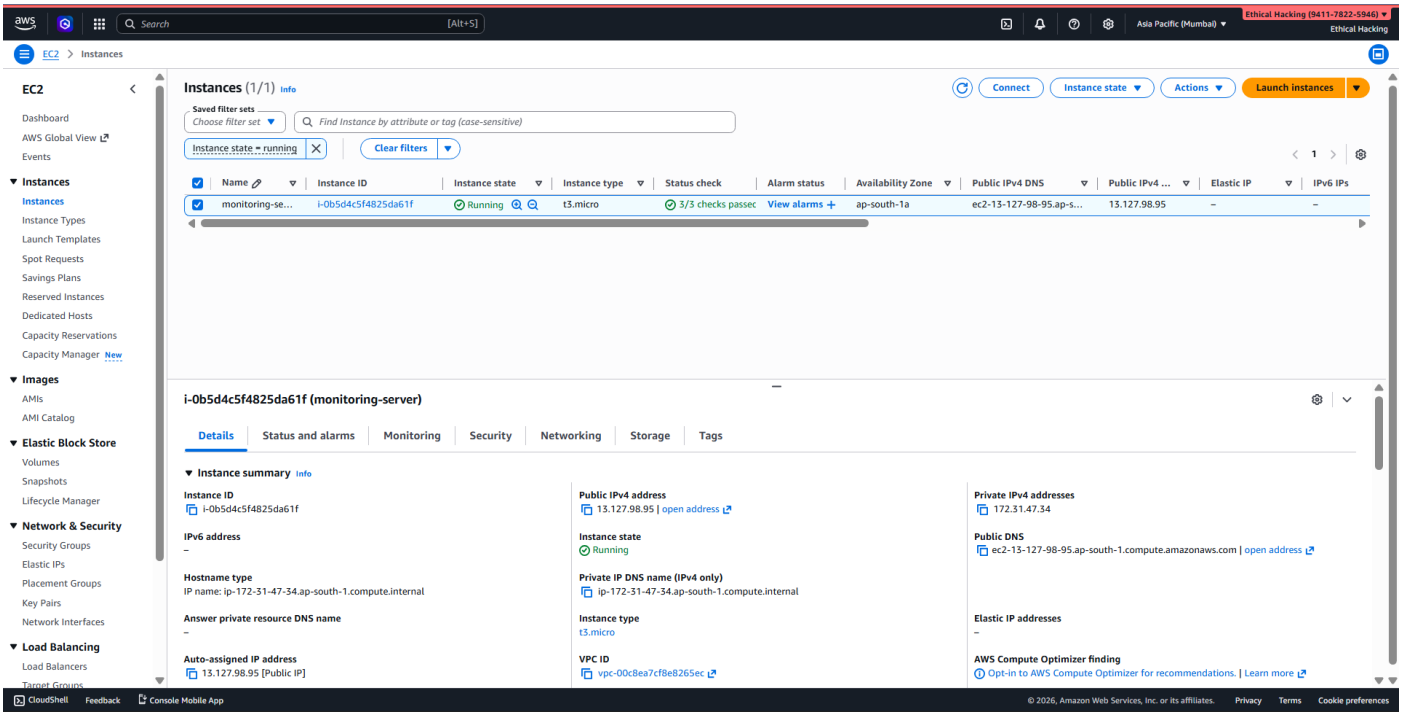
```
              ]
            + from_port        = 9090
            + ipv6_cidr_blocks = []
            + prefix_list_ids  = []
            + protocol         = "tcp"
            + security_groups  = []
            + self             = false
            + to_port          = 9090
              # (1 unchanged attribute hidden)
          },
        + {
            + cidr_blocks      = [
                + "0.0.0.0/0",
              ]
            + from_port        = 9100
            + ipv6_cidr_blocks = []
            + prefix_list_ids  = []
            + protocol         = "tcp"
            + security_groups  = []
            + self             = false
            + to_port          = 9100
              # (1 unchanged attribute hidden)
          },
      ]
      + name                   = "monitoring-sg"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + region                 = "ap-south-1"
      + revoke_rules_on_delete = false
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + grafana_url    = (known after apply)
  + prometheus_url = (known after apply)
  + public_ip      = (known after apply)

─────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$
```

```
GHOST@GHOST:~/Projects/DevOps_Task/Task_Day_19/Monitorin_Task$ terraform apply
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 1s [id=ami-095cc90aa5ddff518]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.monitoring_server will be created
  + resource "aws_instance" "monitoring_server" {
      + ami                                  = "ami-095cc90aa5ddff518"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "terraform-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "ap-south-1"
      + secondary_private_ips                = (known after apply)
      + security_groups                      = [
          + "monitoring-sg",
        ]
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = (known after apply)
      + tags                                 = {
          + "Name" = "monitoring-server"
```

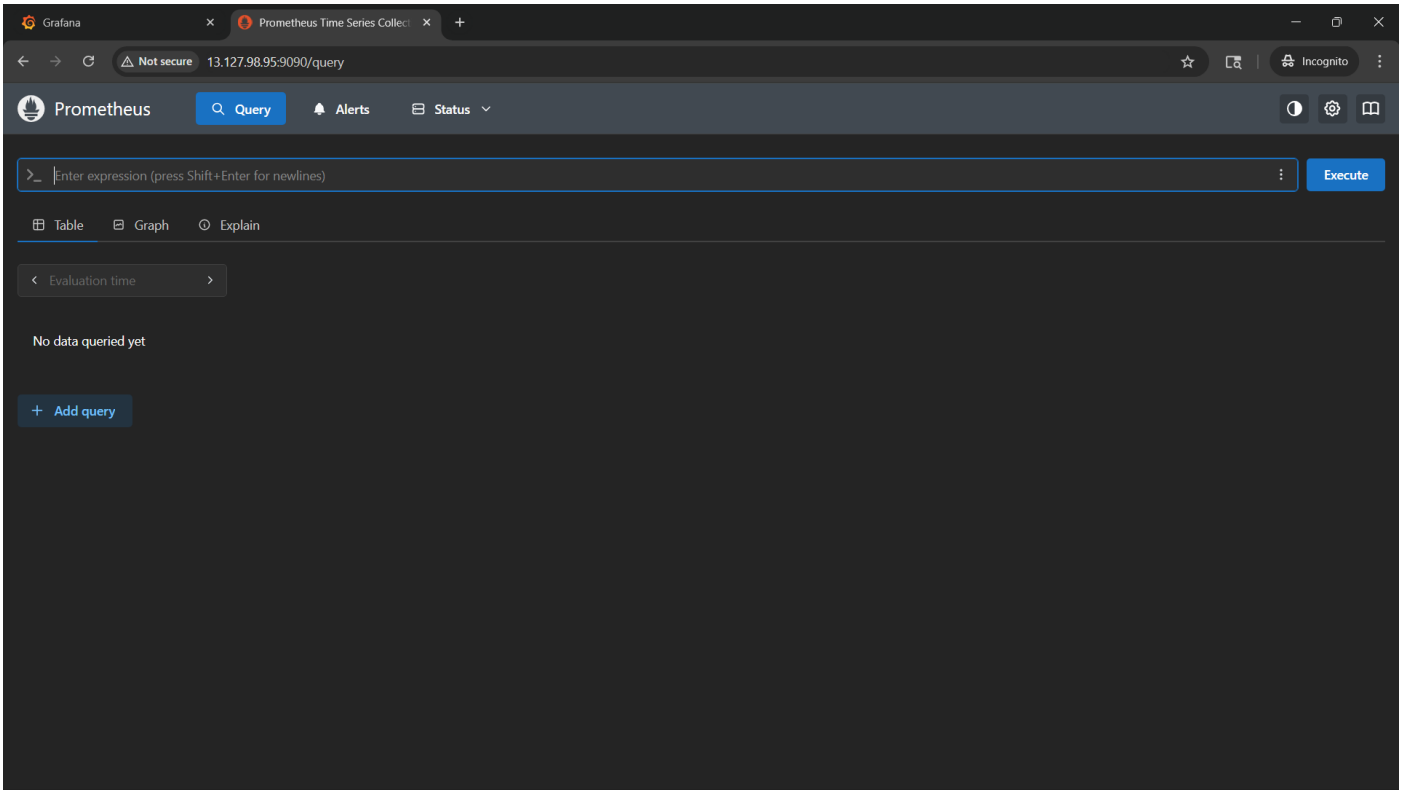
```
            },
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + from_port        = 9100
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 9100
                # (1 unchanged attribute hidden)
            },
        ]
      + name                   = "monitoring-sg"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + region                 = "ap-south-1"
      + revoke_rules_on_delete = false
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + grafana_url    = (known after apply)
  + prometheus_url = (known after apply)
  + public_ip      = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_key_pair.generated_key: Creating...
aws_security_group.monitoring_sg: Creating...
aws_key_pair.generated_key: Creation complete after 1s [id=terraform-key]
aws_security_group.monitoring_sg: Creation complete after 3s [id=sg-06ee0bf6bea853720]
aws_instance.monitoring_server: Creating...
aws_instance.monitoring_server: Still creating... [00m10s elapsed]
aws_instance.monitoring_server: Creation complete after 12s [id=i-0d4071d1ec2924fed]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

After creating the EC2 using Terraform, Verifying the created EC2 and All the services:

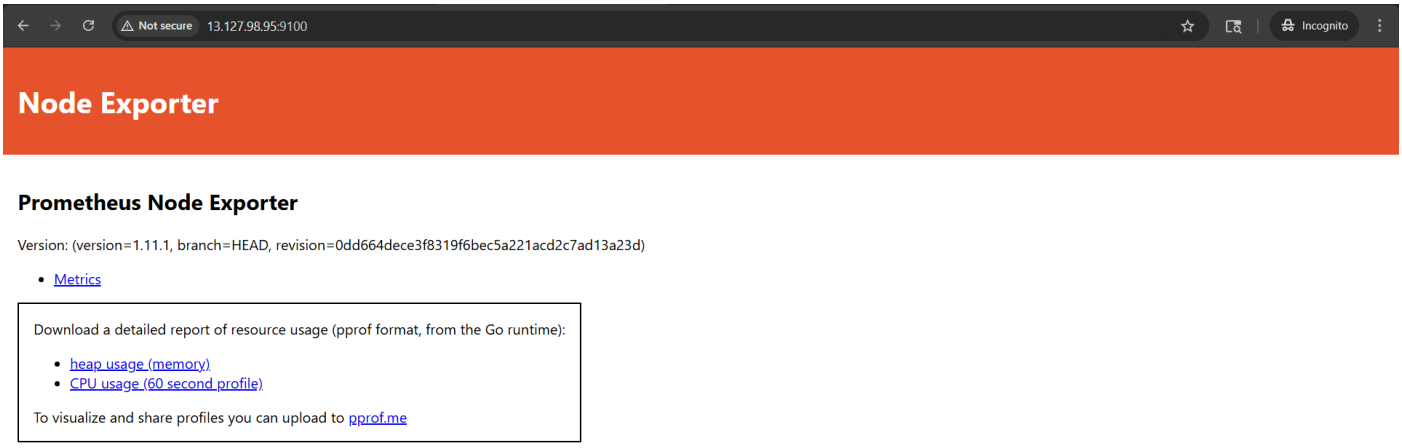
EC2 Created Successfully:



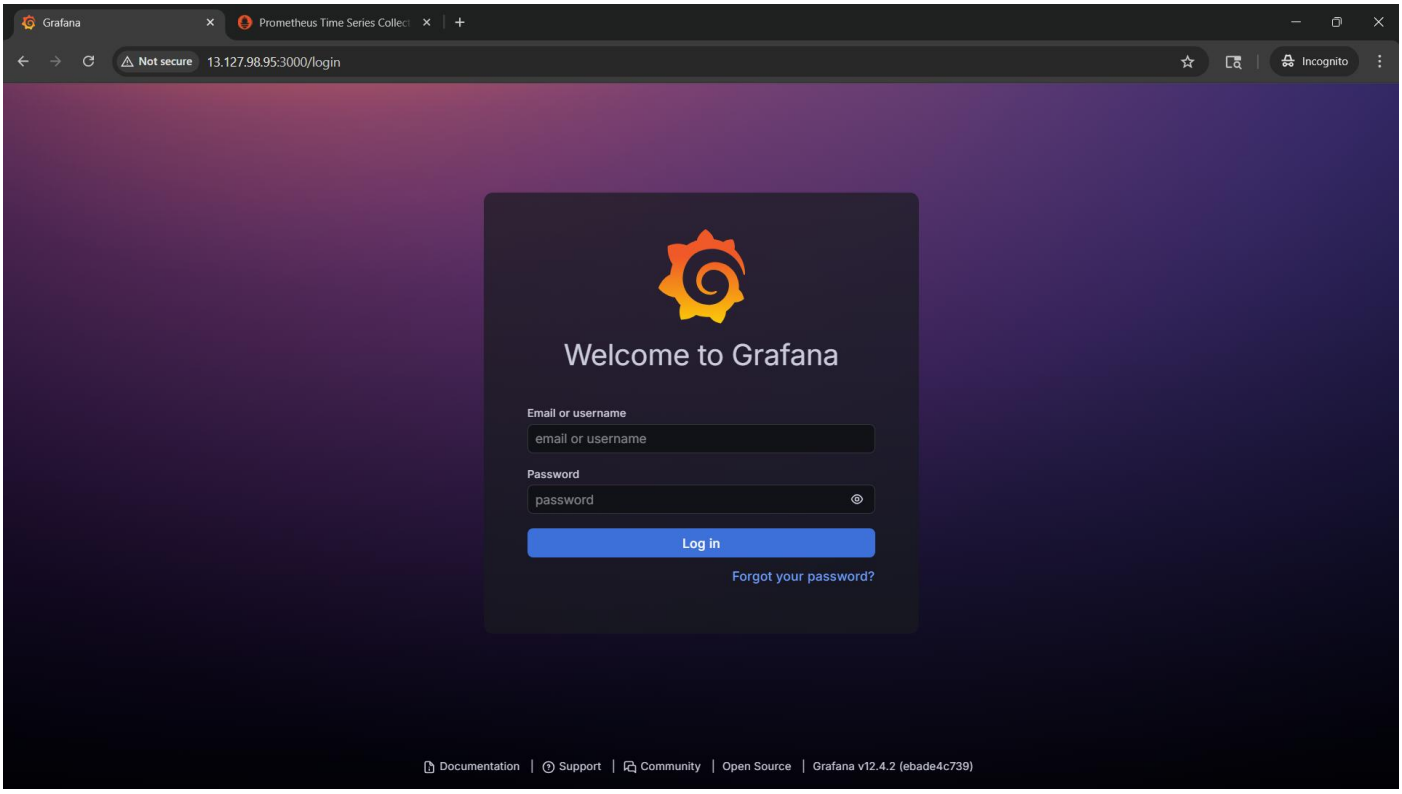
Prometheus Working Fine:



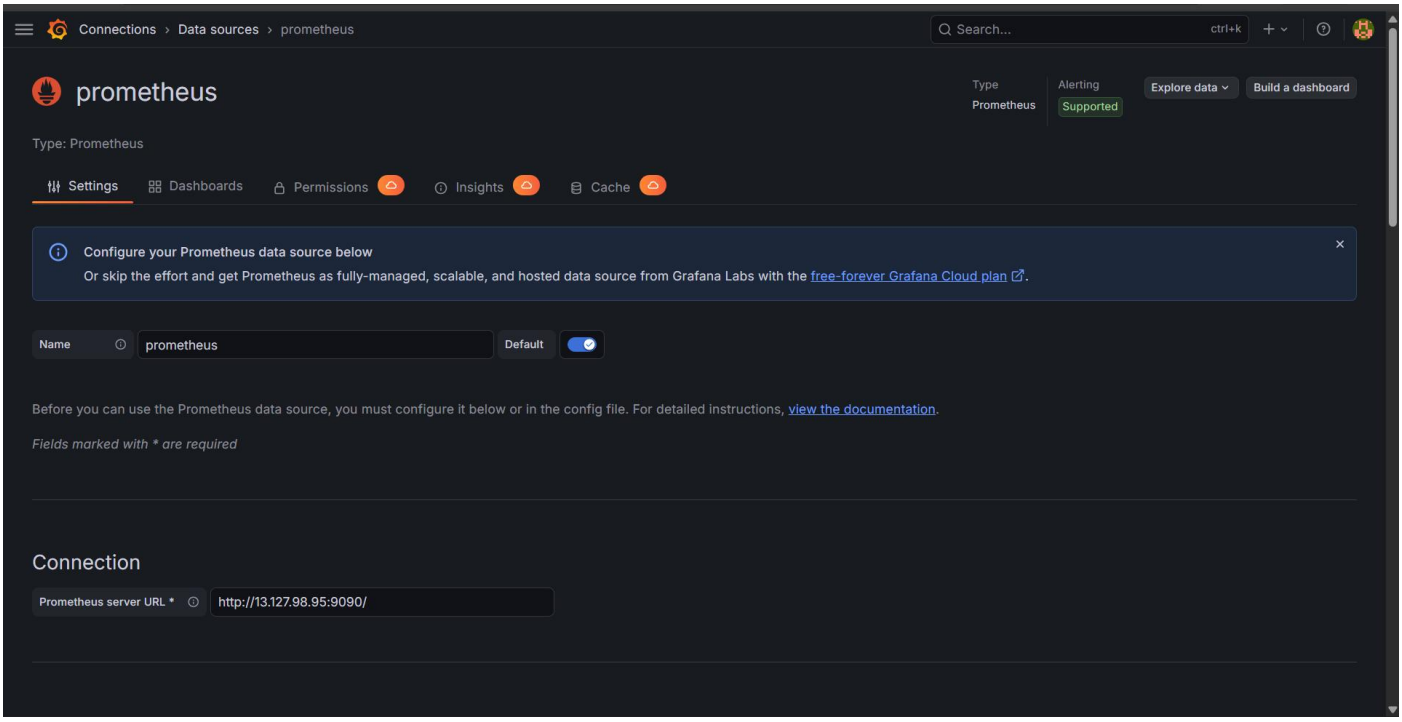
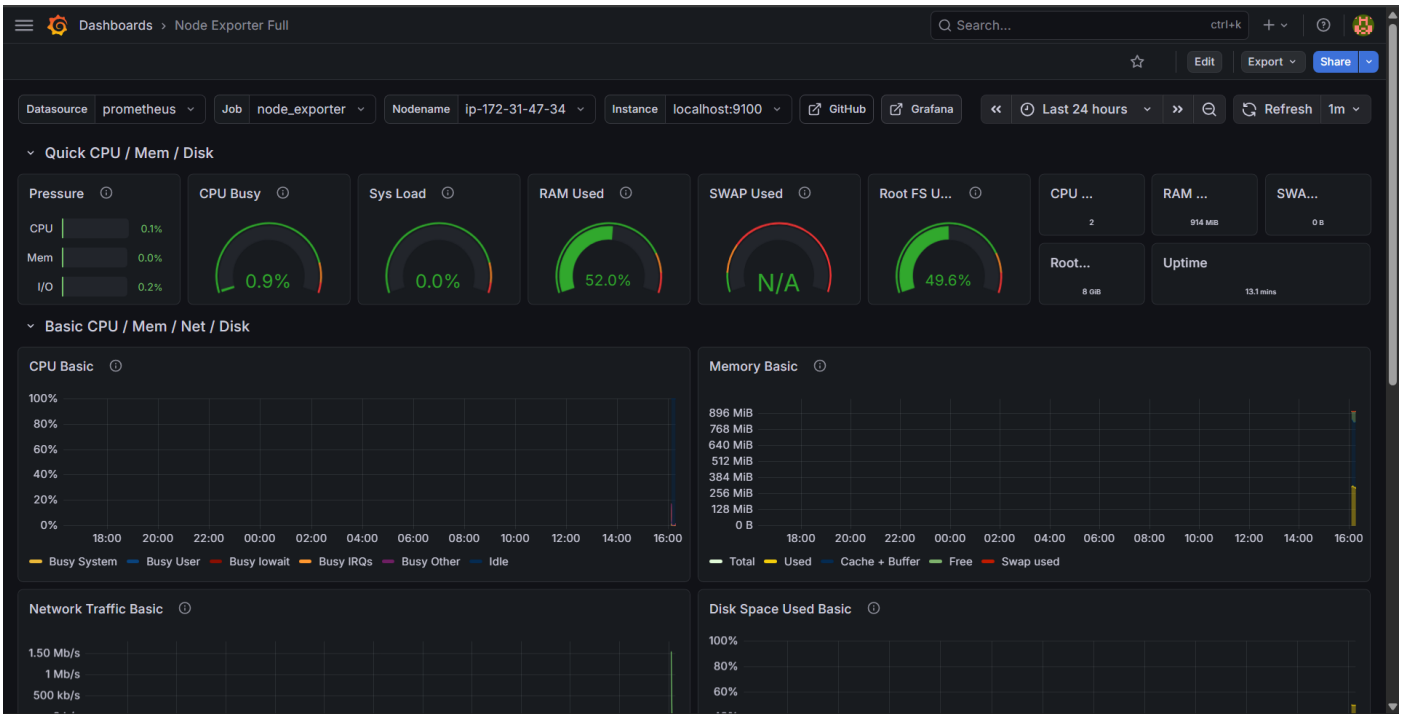
Node Exporter:



Grafana Working Fine:



Connecting Grafana with Prometheus to get the data:



Final Working Dashboard imported from Grafana: